WEEK 6 – MOVIE REVIEW AND RATING MANAGEMENT SYSTEM

Scenario:
The Movie Ticket Booking System allows customers to give feedback about movies they have watched. Customers can provide ratings and reviews after completing their booking. The system stores customer reviews, analyzes movie performance, and identifies highly rated movies.

1.Create Review Table

```
SQL> CREATE TABLE MOVIE_REVIEW (
  2      REVIEW_ID VARCHAR2(7) PRIMARY KEY,
  3      CUSTOMER_ID VARCHAR2(7),
  4      MOVIE_ID VARCHAR2(7),
  5      RATING NUMBER(1) CHECK (RATING BETWEEN 1 AND 5),
  6      REVIEW_COMMENT VARCHAR2(100),
  7      REVIEW_DATE DATE,
  8      FOREIGN KEY (CUSTOMER_ID) REFERENCES CUSTOMER(CUSTOMER_ID)
  9  );

Table created.
```

```
SQL> desc movie_review;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 REVIEW_ID                                 NOT NULL VARCHAR2(7)
 CUSTOMER_ID                                        VARCHAR2(7)
 MOVIE_ID                                           VARCHAR2(7)
 RATING                                             NUMBER(1)
 REVIEW_COMMENT                                     VARCHAR2(100)
 REVIEW_DATE                                        DATE
```

Store Customer Feedback and Ratings

Tasks:

1. Add reviews for purchased products.

```
SQL> INSERT INTO MOVIE_REVIEW VALUES ('R101','S101','M101',5,'Excellent movie',TO_DATE('2026-09-20','YYYY-MM-DD'));
1 row created.
SQL> INSERT INTO MOVIE_REVIEW VALUES ('R102','S102','M102',4,'Very good movie',TO_DATE('2026-09-21','YYYY-MM-DD'));
1 row created.
SQL> INSERT INTO MOVIE_REVIEW VALUES ('R103','S103','M103',3,'Good movie',TO_DATE('2026-09-22','YYYY-MM-DD'));
1 row created.
SQL> INSERT INTO MOVIE_REVIEW VALUES ('R104','S104','M101',5,'Amazing movie',TO_DATE('2026-09-23','YYYY-MM-DD'));
1 row created.
SQL> INSERT INTO MOVIE_REVIEW VALUES ('R105','S105','M104',2,'Average movie',TO_DATE('2026-09-24','YYYY-MM-DD'));
1 row created.
SQL> INSERT INTO MOVIE_REVIEW VALUES ('R106','S106','M102',4,'Nice movie',TO_DATE('2026-09-25','YYYY-MM-DD'));
1 row created.
SQL> INSERT INTO MOVIE_REVIEW VALUES ('R107','S107','M105',5,'Loved the movie',TO_DATE('2026-09-26','YYYY-MM-DD'));
1 row created.
SQL> INSERT INTO MOVIE_REVIEW VALUES ('R108','S108','M103',3,'It was okay',TO_DATE('2026-09-27','YYYY-MM-DD'));
1 row created.
SQL> INSERT INTO MOVIE_REVIEW VALUES ('R109','S109','M101',4,'Good experience',TO_DATE('2026-09-28','YYYY-MM-DD'));
1 row created.
SQL> INSERT INTO MOVIE_REVIEW VALUES ('R110','S110','M105',5,'Fantastic movie',TO_DATE('2026-09-29','YYYY-MM-DD'));
1 row created.
```

Display All Reviews

```
SQL> SELECT * FROM MOVIE_REVIEW;

REVIEW_ID  CUSTOMER_ID  MOVIE_ID   RATING REVIEW_COMMENT           REVIEW_DATE
---------- ------------ ---------- ------ ----------------------- ------------
R101       S101         M101            5 Excellent movie         20-SEP-26
R102       S102         M102            4 Very good movie         21-SEP-26
R103       S103         M103            3 Good movie              22-SEP-26
R104       S104         M101            5 Amazing movie           23-SEP-26
R105       S105         M104            2 Average movie           24-SEP-26
R106       S106         M102            4 Nice movie              25-SEP-26
R107       S107         M105            5 Loved the movie         26-SEP-26
R108       S108         M103            3 It was okay             27-SEP-26
R109       S109         M101            4 Good experience         28-SEP-26
R110       S110         M105            5 Fantastic movie         29-SEP-26

10 rows selected.
```

2.)Store customer ratings from 1 to 5 stars.

Ensure this by adding Constraint

```
SQL> ALTER TABLE MOVIE_REVIEW
  2  ADD CONSTRAINT CHECK_RATING
  3  CHECK (RATING BETWEEN 1 AND 5);

Table altered.
```

```
SQL> INSERT INTO MOVIE_REVIEW VALUES
  2  ('R111','S101','M101',6,'Good movie',TO_DATE('2026-10-01','YYYY-MM-DD'));
INSERT INTO MOVIE_REVIEW VALUES
*
ERROR at line 1:
ORA-02290: check constraint (C##MOVIE_TICKET_BOOKING.CHECK_RATING) violated
```

3. Update review comments if required.

```
SQL> UPDATE MOVIE_REVIEW
  2  SET REVIEW_COMMENT = 'Excellent and entertaining movie'
  3  WHERE REVIEW_ID = 'R101';

1 row updated.
```

3.Retrieve Product Review Details

Write SQL queries to:

1. Display all reviews for a product.

```
SQL> SELECT *
  2  FROM MOVIE_REVIEW
  3  WHERE MOVIE_ID = 'M101';

REVIEW_ID  CUSTOMER_ID  MOVIE_ID   RATING REVIEW_COMMENT           REVIEW_DATE
---------- ------------ ---------- ------ ------------------------ -----------
R101       S101         M101            5 Excellent movie          20-SEP-26
R104       S104         M101            5 Amazing movie            23-SEP-26
R109       S109         M101            4 Good experience          28-SEP-26
```

2.Display Customer Name with Reviews

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.FIRST_NAME,
  3         C.LAST_NAME,
  4         R.MOVIE_ID,
  5         R.RATING,
  6         R.REVIEW_COMMENT
  7  FROM CUSTOMER C
  8  JOIN MOVIE_REVIEW R
  9  ON C.CUSTOMER_ID = R.CUSTOMER_ID;

CUSTOMER_ID  FIRST_NAME      LAST_NAME       MOVIE_ID   RATING REVIEW_COMMENT
------------ --------------- --------------- ---------- ------ -------------------------
S101         Akshaya         Anantharaj      M101            5 Excellent and entertainin
                                                                g movie

S102         Divya           Sri             M102            4 Very good movie
S103         Vimal           Krishna         M103            3 Good movie
S104         Sri             Vatsan          M101            5 Amazing movie
S105         Pooja           Shree           M104            2 Average movie
S106         Reena           Vijay           M102            4 Nice movie
S107         Amala           Arun            M105            5 Loved the movie
S108         Joseph          Vijay           M103            3 It was okay
S109         Roselin         Mary            M101            4 Good experience

CUSTOMER_ID  FIRST_NAME      LAST_NAME       MOVIE_ID   RATING REVIEW_COMMENT
------------ --------------- --------------- ---------- ------ -------------------------
S110         Evangeline      Sheeba          M105            5 Fantastic movie

10 rows selected.
```

3.Find Movies Having Maximum Reviews

```
SQL> SELECT MOVIE_ID,
  2         COUNT(*) AS REVIEW_COUNT
  3  FROM MOVIE_REVIEW
  4  GROUP BY MOVIE_ID
  5  ORDER BY REVIEW_COUNT DESC;

MOVIE_ID   REVIEW_COUNT
---------- ------------
M101                  3
M102                  2
M103                  2
M105                  2
M104                  1
```

4. Display Recent Customer Feedback

```
SQL> SELECT *
  2  FROM (
  3      SELECT *
  4      FROM MOVIE_REVIEW
  5      ORDER BY REVIEW_DATE DESC
  6  )
  7  WHERE ROWNUM <= 5;

REVIEW_ID  CUSTOMER_ID  MOVIE_ID   RATING REVIEW_COMMENT           REVIEW_DATE
---------- ------------ ---------- ------ ------------------------ ------------
R110       S110         M105            5 Fantastic movie          29-SEP-26
R109       S109         M101            4 Good experience          28-SEP-26
R108       S108         M103            3 It was okay              27-SEP-26
R107       S107         M105            5 Loved the movie          26-SEP-26
R106       S106         M102            4 Nice movie               25-SEP-26
```

5.Display Reviews with Rating Above 4

```
SQL> SELECT *
  2  FROM MOVIE_REVIEW
  3  WHERE RATING > 4;

REVIEW_ID  CUSTOMER_ID  MOVIE_ID   RATING REVIEW_COMMENT           REVIEW_DATE
---------- ------------ ---------- ------ ------------------------ ------------
R101       S101         M101            5 Excellent and entertainin 20-SEP-26
                                          g movie

R104       S104         M101            5 Amazing movie            23-SEP-26
R107       S107         M105            5 Loved the movie          26-SEP-26
R110       S110         M105            5 Fantastic movie          29-SEP-26
```

9. Calculate Average Rating for Each Movie

```
SQL> SELECT MOVIE_ID, AVG(RATING) AS AVERAGE_RATING
  2  FROM MOVIE_REVIEW
  3  GROUP BY MOVIE_ID;

MOVIE_I AVERAGE_RATING
------- --------------
M101        4.66666667
M102                 4
M104                 2
M105                 5
M103                 3
```

10. Count Reviews for Each Movie

```
SQL> SELECT MOVIE_ID, COUNT(*) AS NUMBER_OF_REVIEWS
  2  FROM MOVIE_REVIEW
  3  GROUP BY MOVIE_ID;

MOVIE_I NUMBER_OF_REVIEWS
------- -----------------
M101                    3
M102                    2
M104                    1
M105                    2
M103                    2
```

11. Find Highest-Rated Movies

```
SQL> SELECT MOVIE_ID,
  2         AVG(RATING) AS AVERAGE_RATING
  3  FROM MOVIE_REVIEW
  4  GROUP BY MOVIE_ID
  5  ORDER BY AVERAGE_RATING DESC;

MOVIE_I AVERAGE_RATING
------- --------------
M105                 5
M101        4.66666667
M102                 4
M103                 3
M104                 2
```

12. Movies with Average Rating Above 4

```
SQL> SELECT MOVIE_ID,  AVG(RATING) AS AVERAGE_RATING FROM MOVIE_REVIEW GROUP BY MOVIE_ID HAVING AVG(RATING) > 4;

MOVIE_I AVERAGE_RATING
------- --------------
M101        4.66666667
M105                 5
```

13. Update Review

```
SQL> UPDATE MOVIE_REVIEW
  2  SET REVIEW_COMMENT='Excellent and entertaining movie'
  3  WHERE REVIEW_ID='R101';

1 row updated.
```

14. Delete Invalid Review

```
SQL> DELETE FROM MOVIE_REVIEW
  2  WHERE REVIEW_ID='R105';

1 row deleted.
```

15. Movie Rating Analysis Report

```
SQL> SELECT MOVIE_ID,
  2         COUNT(*) AS NUMBER_OF_REVIEWS,
  3         ROUND(AVG(RATING),2) AS AVERAGE_RATING
  4  FROM MOVIE_REVIEW
  5  GROUP BY MOVIE_ID
  6  ORDER BY AVERAGE_RATING DESC;

MOVIE_I NUMBER_OF_REVIEWS AVERAGE_RATING
------- ----------------- --------------
M105                    2              5
M101                    3           4.67
M102                    2              4
M103                    2              3
```

Report 3: Rating Distribution

16. Rating Distribution – 5 Star Ratings

```
SQL> SELECT COUNT(*) AS FIVE_STAR_RATINGS
  2  FROM MOVIE_REVIEW
  3  WHERE RATING=5;

FIVE_STAR_RATINGS
-----------------
                4
```

17. Rating Distribution – 4 Star Ratings

```
SQL> SELECT COUNT(*) AS FOUR_STAR_RATINGS
  2  FROM MOVIE_REVIEW
  3  WHERE RATING=4;

FOUR_STAR_RATINGS
-----------------
                3
```

18. Low-Rated Reviews

```
SQL> SELECT COUNT(*) AS LOW_RATINGS
  2  FROM MOVIE_REVIEW
  3  WHERE RATING <= 2;

LOW_RATINGS
-----------
          0
```

19. Complete Rating Distribution

```
SQL> SELECT RATING,
  2         COUNT(*) AS NUMBER_OF_RATINGS
  3  FROM MOVIE_REVIEW
  4  GROUP BY RATING
  5  ORDER BY RATING DESC;

    RATING NUMBER_OF_RATINGS
---------- -----------------
         5                 4
         4                 3
         3                 2
```

Conclusion:

The Movie Ticket Booking System's Movie Review and Rating Management module was successfully developed using Oracle SQL. It allows customers to submit movie reviews and ratings from 1 to 5 stars. The system supports inserting, updating, deleting, and retrieving reviews. Aggregate functions such as COUNT() and AVG() are used to calculate the number of reviews and average ratings for each movie. The system also identifies highly rated movies and displays the complete rating distribution. Thus, this module helps analyze customer feedback and evaluate movie performance efficiently.